



Prof. Dr. Bernhard Seeger
Manuel Oed

Projects for the lecture
NoSQL-Database Systems

Submission: 30. 11. 2025,
no later than 10:00 Uhr
via ILIAS

Project 1

Note: It is mandatory to submit your solutions in groups of 3 people!

For the submission, create a team in ILIAS with the group members. Then, combine all your files into a single ZIP file and upload it before the deadline.

In this project, you will implement a data structure for versioned data. For one key, we want to store multiple versions of its value. A key-value pair in this setting is described as follows: For each key k , there is a version list in which several values are stored. In the version list, the individual values are each stored with an associated version number. The key with its version list is called an entry and is explained in the following example. The entry $(5 : [(‘ten’, 8), (‘five’, 7), (‘ten’, 2)])$ stores the key 5 together with its version list, which already stores three versions. The value of the key 5 with version number 2 is ‘ten’, with version number 7 is ‘five’, and finally with version number 8 is ‘ten’. The version numbers are drawn atomically, i.e., there is an atomic clock that provides a new version number (or timestamp) on request. This means that each version number exists only once. In our case, we will simply use a counter starting at 1, which is incremented whenever data is appended.

The project will largely be based on [this](#) paper; you should read it for a detailed description of the algorithms.

Task 1.1: Append-Only Linked List as Version List (2 + 8) (10 Points)

Given a timestamp t , one main query on the version lists will be to find the most recent version that was visible at time t (i.e., the most recent version with a timestamp $t' \leq t$). We call this version the *visible version* at time t .

- a) To get started, create a class `VLinkedList<P>`, an append-only linked list that implements the `VersionList<P>` interface. Each node should store a payload of type `P`. A new node is always appended to the head of the list, and, hence, the head of the list is always the node with the newest version (with the highest version number). Do not use a Java Linked list.
- b) Create a class `BackedVLinkedList<P>` that also implements the `VersionList<P>` interface. However, this version list should persist its nodes in a key-value store:
 - Use a `Serializer<P>` (provided interface) to serialize payloads.
 - Each node should store a serialized payload as a `String` instead of type `P`.

- Use a `KVStore` (provided interface) to store serialized nodes. Use each node's version number to create a key (`String`) for that node in the store. We assume that the version number is unique.
- In each node, the pointer to the next node in the list should be the `String` representing the next node's key in the store.

To serialize and deserialize nodes, you may use Jackson's `ObjectMapper`. Note that objects that are deserialized using an `ObjectMapper` need to provide a default constructor, and attributes must either be public or getters must be provided. See `JacksonExample`.

Task 1.2: Frugal Skiplists for Managing Version Lists (3 + 7)

(10 Points)

The Linked Lists implemented in Task 1.1 may perform poorly when querying for a visible version. Hence, we want to implement a data structure for maintaining version lists that solves this task more efficiently. Similar to a Skip List, this list is called a *Frugal Skiplist* and is described in detail in the paper. However, we will provide an overview in the following:

If a new version is inserted, it is always appended to the head of the list. As the version numbers increase globally, the list is always sorted in descending order, and the head always points to the node with the newest version. Each node has a pointer `next` which points to the next smaller version in the list. Additionally, each node has a pointer `vRidgy` to some other node in the list, which allows us to skip nodes while traversing the list to find the visible version at a given time t . An additional attribute `level` is used to set the `vRidgy` pointer accordingly. This is described in detail in Algorithm 1 and Figure 5 in the paper. Algorithm 2 shows how to utilize the `vRidgy` pointers to find the visible version (given some timestamp t).

- Create a class `FrugalSkiplist<P>` implements `VersionList<P>`. Implement the methods `append` and `findVisible` according to the paper.
- Create a class `BackedFrugalSkiplist<P>` that also implements the `VersionList` interface. As in the previous task, ensure that the nodes of this list get serialized and stored using a `KVStore` (see Task 1.1 for more details).

Task 1.3: A simple Multiversion Map (7 + 1 + 2)

(10 Points)

In this task, we want to implement a simple Multi-Version Tree Map. Each key in the tree corresponds to a `VersionList` containing different versions of the payload.

One goal of this data structure will be to efficiently find a snapshot for a range of keys (a *Range-Snapshot*): given a key `low`, a key `high`, and a timestamp t , we want to find the visible version at time t for all records whose key lies between `low` and `high`.

- Implement a class `BackedSimpleMVM<K extends Comparable<? super K>, P>` that implements the `MultiVersionMap` interface. This class should maintain a Java `TreeMap`, which maps keys to their corresponding version lists. Maintain an attribute of type `VersionListFactory<P>`, which is passed to and set in the constructor of `BackedSimpleMVM`. The `VersionListFactory` is used to create a new `VersionList` whenever needed. Use a counter of type `long` that provides a new version whenever a data item is appended (starting at 1).

b) Complete the class `Test`, such that the provided test data set is read and inserted into a

- `BackedSimpleMVM` using `BackedVLinkedList` for version lists,
- `BackedSimpleMVM` using `BackedFrugalSkiplist` for version lists.

Note that the field `timestamp` in the provided data is simply to verify the results, as these should match the version number assigned automatically by your `MultiVersionMap`.

Use an implementation of the (Flushable) `KVStore` interface that uses Jedis to access a local instance of a Redis server, which is used as a key-value store. Query a snapshot ranging from `KEY002` to `KEY004` (both inclusive) at version/timestamp 20 and output the result. The correct output is provided as a comment in the `Test` class. Make sure to *flush* your jedis instance before running a new test.

c) Compare the runtime of a

- `BackedSimpleMVM` using `BackedVLinkedList` for version lists,
- `BackedSimpleMVM` using `BackedFrugalSkiplist` for version lists.

Expand the `Test` class, such that the benchmark data is read from the corresponding file. Measure the time needed to insert the data into the two `MultiVersionMaps`. Additionally, run a full-range snapshot query for the following timestamps: 10, 100, 500, 1_000, 5_000, 10_000, 50_000, 100_000, 500_000. Run the queries (and insertions) several times and report averages to avoid noise. Discuss and explain the results.

Task 1.4: VWeaver Multiversion Map (7 + 1 + 2)

(10 Points)

Now we want to speed up the Range-Snapshot query using the VWeaver approach from the paper. That is, you should extend the functionality of the Frugal Skiplists of the previous tasks by cross-list pointers. More precisely, each node in the extended Frugal Skiplist carries an additional pointer `kRidgy`, which points to a node in the skiplist corresponding to the next relevant key. The idea is that following the `kRidgy` pointer already gets you close to the visible version of the next record, rather than having to start at the head of the next record. You should read about the details in the paper, especially Section 3.2.

- a)** Create a class `BackedVWeaverMVM<K>` extends `Comparable<? super K>`, `P>` which implements the `MultiVersionMap` interface. As before, the nodes of the version list should be serialized and stored utilizing the `KVStore` interface. Make sure that the `append` method sets the `kRidgy` pointer, and that the `rangeSnapshot` method utilizes the `kRidgy` pointers according to the paper.
- b)** Finally, test the runtime of `BackedVWeaverMVM` in the same setup as in the previous task and discuss the results.
- c)** Explain (in theory) an idea of how the Range-Snapshot queries in `BackedVWeaverMVM` could be made more efficient, assuming you had access to the internals of the Search Tree (or an appropriate method would be exposed). For simplicity, assume that each node in the tree has a pointer to its parent node (except, of course, the root).